

Function-Aware Fill-in-the-Middle as Mid-Training for Coding Agent Foundation Models

Yubo Wang, Jiarong Liang, Yuxuan Zhang, Xuye Liu, Cong Wei, Yuyu Zhang, Ping Nie, Wenhui Chen

ABSTRACT

Coding agents must integrate external tool returns into ongoing reasoning - a capability that standard left-to-right pretraining on code exposes only in its forward direction. We observe that the action-observation-continuation loop of a coding agent is structurally isomorphic to a function call site, where a caller binds arguments, a callee returns a value computed elsewhere, and downstream code consumes that value. This conditioning structure exists at internet scale in ordinary code. We exploit it through function-aware fill-in-the-middle (FIM) mid-training: a self-supervised objective that masks functions selected via program dependency graph analysis and a complexity-inferability double criterion. We mid-train Qwen2.5-Coder-Instruct (7B/14B) and Qwen3-8B on a 2.6B-token decontaminated corpus drawn from 968 GitHub repositories, then apply existing agentic post-training pipelines. Mid-training improves SWE-Bench-Verified by +2.8/+3.0 at 7B/14B and by +3.2 on Qwen3-8B; SWE-Bench-Lite gains are +3.7/+4.0/+5.4 on the same models. The improvement holds across two post-training pipelines (R2E-Gym, SWE-Smith) and on a non-Qwen2.5 base (Qwen3-8B with SWE-Lego). Beyond in-domain gains, mid-training also mitigates the capability erosion that agentic post-training otherwise inflicts on non-agent coding (e.g., LiveCodeBench) and non-coding tool-use benchmarks (tau-bench, BFCL): although the mid-training corpus contains Python code only, the function-call inductive bias survives post-training and yields consistent gains.

About this document

This is a **Reviewed Preprint**: a third-party paper that llmXive has *auto-reviewed* (with its LLM review panel) but *never modified*. The original manuscript follows this cover page **exactly as published** by its authors — llmXive re-typesets nothing and claims no authorship.

Provenance. Ingested by llmXive on 2026-07-16 — scraped from arXiv; via llmXive dashboard, GitHub issue #1235; submitted by github-actions[bot].

Source. <https://arxiv.org/abs/2607.12463>

About llmXive. llmXive is an automated platform for open scientific discovery. Its specialist LLM agents advance their own ideas from brainstorm to peer-reviewed paper, and they also review notable third-party papers — drawn from the most-downloaded papers on Hugging Face and from submissions by human contributors. Every submitted paper is reviewed by the LLM panel *and* used to seed a new llmXive follow-up study. This paper is one such third-party work: llmXive reviewed it but did not write it. Learn more at <https://github.com/ContextLab/llmXive>.

Automated review. See the accompanying [peer-review-llmxive.pdf](#) for the full reviewer report.

llmXive follow-up. A separate llmXive study building on this work: [PROJ-1074-llmxive-follow-up-extending-function-awa](#).

Function-Aware Fill-in-the-Middle as Mid-Training for Coding Agent Foundation Models

Yubo Wang^{1,5*} Jiarong Liang^{1*} Yuxuan Zhang² Xuye Liu¹ Cong Wei^{1,3}
Yuyu Zhang⁴ Ping Nie¹ Wenhui Chen^{1,5*}

¹University of Waterloo, ²University of British Columbia, ³NVIDIA, ⁴Verdent AI, ⁵Vector Institute

<https://github.com/TIGER-AI-Lab/FIM-Midtraining>

Abstract

Coding agents must integrate external tool returns into ongoing reasoning—a capability that standard left-to-right pretraining on code exposes only in its forward direction. We observe that the *action* $\rightarrow$ *observation* $\rightarrow$ *continuation* loop of a coding agent is structurally isomorphic to a function call site, where a caller binds arguments, a callee returns a value computed elsewhere, and downstream code consumes that value. This conditioning structure exists at internet scale in ordinary code. We exploit it through *function-aware fill-in-the-middle (FIM) mid-training*: a self-supervised objective that masks functions selected via program dependency graph analysis and a complexity–inferability double criterion. We mid-train Qwen2.5-Coder-Instruct (7B/14B) and Qwen3-8B on a 2.6B-token decontaminated corpus drawn from 968 GitHub repositories, then apply existing agentic post-training pipelines. Mid-training improves SWE-Bench-Verified by +2.8/ +3.0 at 7B/14B and by +3.2 on Qwen3-8B; SWE-Bench-Lite gains are +3.7/ +4.0/ +5.4 on the same models. The improvement holds across two post-training pipelines (R2E-Gym, SWE-Smith) and on a non-Qwen2.5 base (Qwen3-8B with SWE-Lego). Beyond in-domain gains, mid-training also mitigates the capability erosion that agentic post-training otherwise inflicts on non-agent coding (e.g., LiveCodeBench) and non-coding tool-use benchmarks (τ -bench, BFCL): although the mid-training corpus contains Python code only, the function-call inductive bias survives post-training and yields consistent gains.

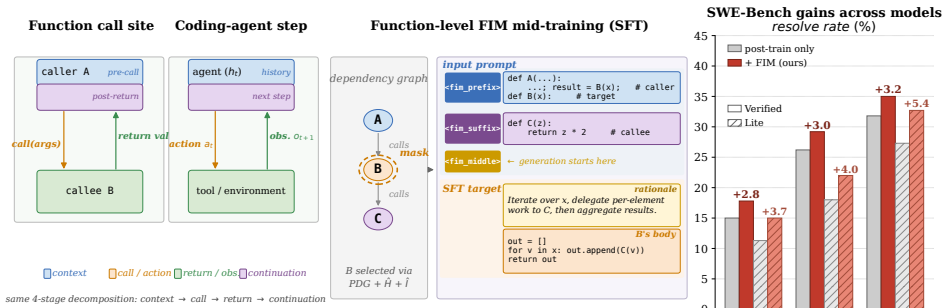


Figure 1: **Left:** A function call site and a single step of a coding agent are *structurally similar*, decomposing into the same four stages: context, call/action, return/observation, continuation. **Middle:** We exploit this analogy via function-aware FIM mid-training. A function B is selected from the program dependency graph using complexity ($\hat{H}$) and inferability ($\hat{I}$) scores; the model is then mid-trained to fill in B 's body together with a CoT rationale, given the surrounding file as an FIM-formatted prompt. **Right:** Mid-training yields consistent gains across both Qwen2.5-Coder-Instruct (7B, 14B) and Qwen3 (8B) on SWE-Bench-Verified (solid bars) and SWE-Bench-Lite (hatched bars).

¹Equal contribution. Corresponding authors: yubo.wang.sunny@gmail.com, hustchenwenhu@gmail.com

1 Introduction

Coding agents that resolve real software engineering issues have moved from research demos to deployed systems [23, 43, 37]. Their progress, however, has been driven almost entirely by scaling synthetic agent-trajectory data during post-training: pipelines such as SWE-Gym [31], R2E-Gym [22], and SWE-Smith [44] curate or synthesize trajectories that imitate human or LLM behavior on issue-resolution tasks. The base model these pipelines start from is typically a code LLM trained with next-token prediction (and, in some cases, random-span FIM) on internet-scale code [20, 14, 24]. Between these two stages lies a training-time gap: the base model is rarely optimized for the conditioning structure that agentic post-training will later demand. We treat this gap as an opportunity for a dedicated *mid-training* stage that aligns the base model with agent-relevant inductive biases before agent-specific data is introduced.

Our central observation is that the inductive bias required by a coding agent already exists in ordinary code, but in a shape that left-to-right pretraining systematically under-exposes. At each step an agent maintains a history h_t , samples an action $a_t \sim \pi(a_t | h_t)$, receives an observation o_{t+1} produced by an external process, and continues conditioned on the entire trace. This four-part decomposition—context, action, externally-computed return, continuation—is precisely the decomposition of a function call site: pre-call code that establishes intent and binds arguments; the call itself; a return value produced by code outside the immediate scope; and downstream code that consumes the return value (Figure 1, left). A model trained to reason *bidirectionally* about function-level dependencies must learn to reconstruct a callee’s behavior from caller context and downstream usage, which is the same competence required to predict an agent’s continuation given a history and a tool return. The correspondence is structural rather than literal—FIM training conditions on a given suffix while agent inference generates one—but it suggests that representations induced by the former should transfer to the latter, an empirical question we address in Section 3.

The fill-in-the-middle objective is not new [2]; recent code LLMs mix random-span FIM into pretraining [20, 14, 24]. Random-span FIM is nonetheless poorly aligned with agentic conditioning for three reasons. (i) *Span boundaries are syntactically arbitrary*: most masked spans cut through expressions or partial statements and carry weak signal about function-level dependencies. (ii) *There is no reasoning supervision*: the model fills the span directly, with no intermediate rationale mirroring an agent’s think-then-act pattern. (iii) *The objective is dissolved into pretraining*: by the time post-training begins, any FIM-conferred structural prior has been amortized across trillions of unrelated tokens. We address all three points: masking targets are selected at function granularity via program dependency graph analysis with a base-model-agnostic complexity–inferability double criterion (Section 2.3); chain-of-thought rationales are embedded *inside* the FIM middle span so the model produces reasoning consistent with the eventual code (Section 2.4); and the objective is applied at a dedicated mid-training stage, concentrating its signal immediately before agentic post-training.

We evaluate this recipe along three axes of robustness. *On the Qwen2.5-Coder-Instruct series*, mid-training improves SWE-Bench-Verified by +2.8 and +3.0 points at 7B and 14B respectively, indicating that the structural prior is not absorbed by larger pretrained models in the practical deployment range. *Across two post-training pipelines*, mid-training improves both R2E-Gym and SWE-Smith on the same 7B base, with the SWE-Smith pairing yielding +5.3 points on SWE-Bench-Verified. *On a non-Qwen2.5 base*, mid-training transfers to Qwen3-8B (paired with SWE-Lego) for a +3.2-point gain on SWE-Bench-Verified; this single comparison varies the post-training pipeline jointly with the base model and should be read as evidence that the prior is not specific to a single Qwen2.5-Coder + R2E-Gym/SWE-Smith combination, rather than as a guarantee across families.

A second set of results probes our motivating hypothesis. All checkpoints are evaluated after the full pipeline (post-training alone, or mid-training followed by post-training), since FIM-only checkpoints degrade instruction-following and are not comparable to instruction-tuned baselines. Agentic post-training alone substantially regresses non-target capabilities at 14B, dropping LiveCodeBench, BFCL, and τ -bench by double-digit margins in some cases; adding mid-training before the same post-training restores +11.1 on LiveCodeBench, +2.4 on BFCL, and +3.9 on τ -bench. Since the mid-training corpus contains only Python code with no tool-use data, the cross-domain recovery is direct evidence for the function-call/tool-call isomorphism.

Contributions. (1) Framing function call sites as the internet-scale analogue of the agent action–observation–continuation loop, motivating function-granularity FIM as a self-supervised prior for

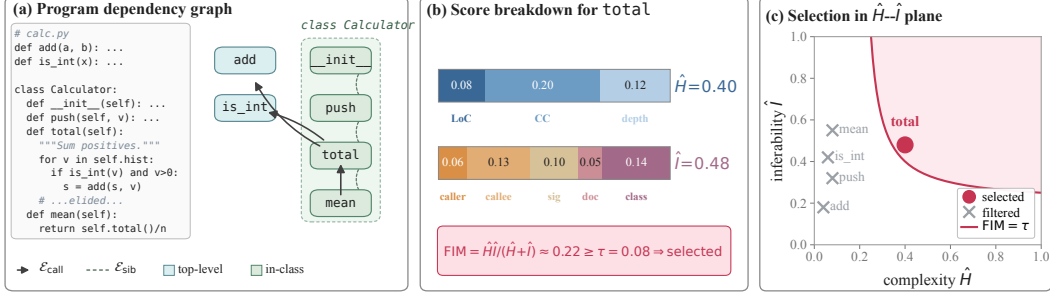


Figure 2: Function-aware FIM target selection on a small calculator example. **(a)** Program dependency graph parsed from the AST: solid arrows are call edges $\mathcal{E}_{\text{call}}$, dashed lines are sibling edges $\mathcal{E}_{\text{sib}}$ between same-class methods. **(b)** Stacked bars decompose the complexity score $\hat{H} = 0.40$ (Eq. 1; LoC, CC, depth) and the inferability score $\hat{I} = 0.48$ (Eq. 2; five context signals) for `Calculator.total`, yielding $\text{FIM} \approx 0.22 \geq \tau = 0.08$ (a hyperparameter; see Appendix B for the full list).

agent capability. **(2)** A function-aware FIM mid-training pipeline combining program dependency graph analysis, a base-model-agnostic complexity–inferability double criterion, and CoT rationales embedded inside the FIM middle span. **(3)** Robustness validation along three axes—two model sizes (Qwen2.5-Coder-Instruct 7B/14B), two post-training pipelines (R2E-Gym, SWE-Smith), and one alternative base (Qwen3-8B with SWE-Lego)—with consistent in-domain gains in every configuration. **(4)** A direct test of the motivating hypothesis: the same Python-only corpus also yields gains on τ -bench, BFCL, and LiveCodeBench, evidence that the function-call inductive bias survives post-training and transfers across task families. **(5)** Open release of the 968-repository decontaminated corpus (400K FIM samples, 2.6B tokens), the selection pipeline, and mid-training checkpoints.

2 Method

2.1 Motivation: Function Calls as Agent-Like Structures

A coding agent at step t samples $a_t \sim \pi(a_t | h_t)$, observes $o_{t+1} \sim p(o_{t+1} | h_t, a_t)$, and continues. Function calls mirror this loop (Figure 1, left): pre-call context, call, return, and downstream usage align with history, action, observation, and continuation. This isomorphism motivates a fill-in-the-middle (FIM) objective drawn from code. Random-span FIM [20, 14, 24] captures it only incidentally; our *function-aware* variant selects masking targets by program structure and contextual predictability.

2.2 Data Collection and Decontamination

We curate a corpus of 968 Python repositories from GitHub. Starting from $\sim 2,000$ candidates retrieved by combining a star-count threshold with ten topic queries, we apply manual quality filtering and remove every repository whose origin overlaps with the source repositories of SWE-Bench [23] (verified by repository name and any known fork). To eliminate test-time leakage, we restrict each repository to commits whose timestamp precedes the earliest base-commit used in SWE-Bench-Verified and SWE-Bench-Lite. The filtered corpus yields $\approx 400\text{K}$ FIM samples ($\approx 2.6\text{B}$ tokens under the Qwen2.5-Coder tokenizer): 320K single-function, 60K pair, and 20K triple targets. Full statistics, topic-category breakdown, and license inventory are reported in Appendix A.

2.3 Function-Aware FIM Target Selection

Given each source file, our pipeline produces a set of mask targets together with the corresponding masked file. Each target is a single function or a connected group of 2–3 functions; the multi-function variant is studied in Section 3.4. The pipeline has four stages: dependency-graph construction, complexity scoring, inferability scoring, and threshold-based selection. Figure 2 illustrates these stages on a small running example—a `Calculator` class with two top-level helpers. The full single-function selection algorithm is given in Appendix B.1, the multi-function extension in Appendix B.7, and a numerical walkthrough of every quantity shown in Figure 2(b) in Appendix B.6.

2.3.1 Program Dependency Graph

For each file we parse its AST and extract the set $\mathcal{V}$ of function nodes (top-level functions and class methods, identified by qualified names). Two edge sets are constructed: *call edges* $\mathcal{E}_{\text{call}}$ between caller and callee, and *sibling edges* $\mathcal{E}_{\text{sib}}$ between methods of the same class (capturing intra-class coupling that flows through shared instance state rather than direct calls). Call resolution handles common Python idioms (direct invocations, class instantiation, `self/cls` method calls) with a short-name fallback against the qualified-name index; Appendix B.2 details the procedure.

2.3.2 Complexity Score $\hat{H}$

For each $v \in \mathcal{V}$ we define

$$\hat{H}(v) = w_\ell \phi(\text{LoC}(v), c_\ell) + w_c \phi(\text{CC}(v), c_c) + w_d \phi(\text{D}(v), c_d), \quad (1)$$

where $\text{LoC}(v)$ is lines of code, $\text{CC}(v)$ is McCabe cyclomatic complexity, $\text{D}(v)$ is the maximum nesting depth of control-flow constructs, and $\phi(x, c) = \min(x/c, 2)$ normalizes each quantity by a soft cap. Caps and weights are listed in Appendix B.3.

2.3.3 Inferability Score $\hat{I}$

A target should be *recoverable* from the surrounding context. $\hat{I}(v)$ aggregates five context-derived signals that approximate the mutual information between v 's body and the rest of the file:

$$\hat{I}(v) = \alpha C_{\text{caller}}(v) + \beta C_{\text{callee}}(v) + \gamma C_{\text{sig}}(v) + \delta C_{\text{doc}}(v) + \varepsilon C_{\text{class}}(v). \quad (2)$$

C_{caller} scores call-site argument specificity, C_{callee} counts intra-file functions called by v , C_{sig} aggregates type annotations and name descriptiveness, C_{doc} indicates docstring presence, and C_{class} counts in-class siblings; full component formulas and weights are in Appendix B.4. Each component is a hand-designed proxy: a learned predictability score would couple selection to a particular reference model and complicate the cross-base-model generalization analysis (Section 3.2).

2.3.4 Single-Function Score

We combine $\hat{H}$ and $\hat{I}$ in a harmonic-mean-like form, scaled by a one-sided difficulty penalty $\rho(\Delta(v))$:

$$\text{FIM}(v) = \frac{\hat{H}(v) \hat{I}(v)}{\hat{H}(v) + \hat{I}(v) + \epsilon} \cdot \rho(\Delta(v)). \quad (3)$$

The harmonic-mean form forces both $\hat{H}$ and $\hat{I}$ to be large simultaneously, penalizing imbalance; ρ down-weights targets that remain hard even given full context, which would otherwise be unlearnable noise. Hard filters on length and dunder methods, the form of ρ , and the threshold $\tau = 0.08$ used throughout this work are documented in Appendix B.5.

2.3.5 Multi-Function Group Selection

Real-world code patches frequently span multiple related functions [23], motivating an extension that masks *groups* of $k=2$ or $k=3$ structurally connected functions; we use this variant in our main recipe and ablate it in Section 3.4. A group score $\text{FIM}(G)$ multiplies a coupling term, the harmonic-mean-like product of group-level $\hat{H}(G)$ and $\hat{I}(G)$, and a difficulty penalty, with $\hat{I}(G)$ recomputed under joint masking so that intra-group references cannot inflate the score. The eight topology patterns (caller-callee, co-callee, sibling-coupled, mutual-call, call-chain, hub, fan-in, class-triad), the full equations, Algorithm 2, and a worked pair example are in Appendix B.7.

2.4 Chain-of-Thought Augmentation

For each selected target we run a three-stage pipeline. **Generate.** Gemini-3-Flash sees only the masked file and produces a step-by-step rationale together with a candidate function body, with no access to the ground-truth body (Appendix B.9). **Filter.** A separate Gemini-3-Flash judge scores the (rationale, candidate body) pair against the ground-truth body on feasibility and five quality dimensions; we keep the top-scoring $\sim 400\text{K}$ samples (Appendix B.10). **Format.** Each retained pair is placed inside the FIM middle span, rationale before body (Appendix B.11):

<fim_prefix> <prefix> <fim_suffix> <suffix> <fim_middle> <rationale> <body>

The model is thus trained to produce reasoning followed by consistent code, mirroring the think-then-act structure of an agent step. The ground-truth body serves only as a filter anchor and never appears in the training target. Section 3.4 isolates the contribution of FIM structure from CoT distillation via a *self-CoT* variant in which the model under training generates its own rationales.

3 Experiments

3.1 Setup

Benchmarks. We evaluate on three groups. *Coding-agent benchmarks:* SWE-Bench-Verified (500) and SWE-Bench-Lite (300) [23], our primary in-domain target. *Non-agent coding benchmarks:* LiveCodeBench [21], OJBench [38], and FullStackBench-EN [5]; these probe pure code generation and serve as a regression check. *Tool-use and OOD agent benchmarks:* Terminal-Bench 2.0 [28], τ -bench [45], and BFCL [32]; the latter two contain no Python code-editing trajectories and test whether the function-call inductive bias transfers across task families.

Mid-training and post-training pipeline. We mid-train Qwen2.5-Coder-Instruct (7B/14B) [20] and Qwen3-8B [42] on the selected FIM corpus using the standard FIM loss on the middle span (rationale plus body), packed to the model’s native context length and using its native FIM sentinel tokens. We then apply existing agentic post-training pipelines without modification: R2E-Gym [22] or SWE-Smith [44] for the Qwen2.5-Coder runs, and SWE-Lego [35] for Qwen3-8B. For Qwen3-8B we train SWE-Lego for 2 epochs rather than the official 4 to prevent overfitting in our setup. Hyperparameters and token budgets are in Appendix C.

Evaluation protocol. All numbers are means over three independent evaluation seeds on the final checkpoint of each pipeline, with “%” omitted in table cells and std bands in parentheses. *Baseline* is base + post-training; *ours* is base + FIM mid-training + identical post-training. We do not evaluate mid-training-only checkpoints because FIM-only models have degraded instruction-following and cannot be compared fairly with instruction-tuned baselines—every reported gain therefore *survives* subsequent post-training. The agent harness is fixed by the post-training pipeline: R2E-Gym uses an OpenHands fork [37, 22], SWE-Smith uses SWE-agent [43], and the Qwen3-8B runs use OpenHands directly (SWE-Lego data exceeds the 32K context of the Qwen2.5-Coder models).

Models and baselines. For Qwen2.5-Coder-Instruct at 7B/14B [20] we report (i) the instruction-tuned base, (ii) post-training with R2E-Gym reproduced under our setup, (iii) the official R2E-Gym numbers (in grey, where available), (iv) our recipe of FIM mid-training followed by R2E-Gym post-training, and at 7B additionally the analogous SWE-Smith [44] variants. Qwen3-8B uses the SWE-Lego [35] pipeline; the cross-base-model comparison thus varies the post-training pipeline simultaneously, a confound we discuss in Section 3.2.

3.2 Main Results on SWE Agent Benchmarks

Table 1 summarizes in-domain agent results.

Consistent gains on the Qwen2.5-Coder-Instruct series. Holding the post-training pipeline fixed at R2E-Gym, FIM mid-training improves SWE-Bench-Verified by +2.80 on 7B-Instruct and +3.00 on 14B-Instruct, with matching directional gains on SWE-Bench-Lite (+3.67/ + 4.00). Both Qwen2.5-Coder-Instruct sizes benefit from the same mid-training corpus and recipe, indicating that the structural prior is not absorbed by the larger pretrained checkpoint within this family.

Transfer across post-training pipelines. Replacing R2E-Gym with SWE-Smith on the same 7B base yields +5.30 points on Verified, larger than the +2.80 under R2E-Gym; on Lite the SWE-Smith pairing gains only +0.50, smaller than +3.67 for the R2E-Gym pairing. The two pipelines together indicate that mid-training is not tuned to a single post-training data distribution, though the magnitude of its benefit depends on the baseline pipeline it composes with.

Transfer to a non-Qwen2.5 base. Switching to Qwen3-8B paired with SWE-Lego, mid-training improves Verified by +3.20 and Lite by +5.40, comparable to the Qwen2.5-Coder-Instruct gains. This single comparison varies the post-training pipeline jointly with the base model, so the result should be read as “not specific to the Qwen2.5-Coder-Instruct + R2E-Gym/SWE-Smith pairing” rather than as a guarantee across base-model families.

Table 1: Main results on coding agent benchmarks. All numbers are percentages, means over three independent seeds with std bands; *Average* is the unweighted mean of Verified and Lite. Bolded rows are our recipe; rows tagged *officially reported* are quoted from the corresponding original publications. SWE-Lego is post-trained for 2 epochs (vs. 4 in the official release) to prevent overfitting.

Setting	SWE-Bench-Verified	SWE-Bench-Lite	Average
<i>Qwen2.5-Coder-7B-Instruct</i>			
— (no agentic training)	1.80 (± 1.30)	1.00 (± 1.00)	1.40
+ R2E-Gym [22] (reproduced)	15.00 (± 1.50)	11.33 (± 1.20)	13.17
+ R2E-Gym (officially reported)	19.00 (± 1.00)	11.00 (± 0.80)	15.00
+ FIM-Midtrain + R2E-Gym	17.80 (± 1.40)	15.00 (± 1.10)	16.40
Δ (ours vs. reproduced)	+2.80	+3.67	+3.24
+ SWE-Smith [44] (reproduced)	12.30 (± 1.20)	14.20 (± 1.40)	13.25
+ SWE-Smith (officially reported)	15.20	11.70	13.45
+ FIM-Midtrain + SWE-Smith	17.60 (± 1.30)	14.70 (± 1.00)	16.15
Δ (ours vs. reproduced)	+5.30	+0.50	+2.90
<i>Qwen2.5-Coder-14B-Instruct</i>			
— (no agentic training)	4.00 (± 1.60)	2.70 (± 1.00)	3.35
+ R2E-Gym (reproduced)	26.20 (± 1.40)	18.00 (± 1.10)	22.10
+ R2E-Gym (officially reported)	26.80 (± 1.40)	20.67 (± 0.70)	23.74
+ FIM-Midtrain + R2E-Gym	29.20 (± 1.50)	22.00 (± 1.20)	25.60
Δ (ours vs. reproduced)	+3.00	+4.00	+3.50
<i>Qwen3-8B</i>			
— (no agentic training)	7.60 (± 1.20)	5.80 (± 0.90)	6.70
+ SWE-Lego [35] (reproduced)	31.80 (± 1.00)	27.30 (± 1.10)	29.55
+ FIM-Midtrain + SWE-Lego	35.00 (± 1.50)	32.70 (± 1.30)	33.85
Δ (ours vs. reproduced)	+3.20	+5.40	+4.30

Table 2: Capability preservation and cross-domain transfer at 14B with R2E-Gym. Bold marks the better trained variant per column (post-only vs. ours); the Instruct row is shown as a reference ceiling. All cells are percentages. “Terminal” denotes Terminal-Bench 2.0.

Setting	Non-agent coding			Agent OOD	Tool use		
	LiveCode	OJBench	FSB-EN	Terminal	τ -bench	BFCL	Avg
Instruct Model	37.20	5.20	53.80	0.00	5.70	23.20	20.85
+ R2E-Gym	24.10	2.80	47.72	2.41	3.40	15.80	16.04
+ FIM Mid-Train + R2E-Gym	35.20	4.74	48.25	3.66	7.30	18.20	19.56
Δ (vs. R2E-Gym only)	+11.10	+1.94	+0.53	+1.25	+3.90	+2.40	+3.52

3.3 Capability Preservation and Cross-Domain Transfer

A natural concern with task-specialized post-training is that it erodes capabilities the base model already had. We further evaluate the 14B model on six additional benchmarks to assess capability preservation. To save compute, we restrict this controlled comparison (instruct base vs. post-training only vs. mid-training + post-training) to R2E-Gym as the post-training dataset (Table 2).

Agentic post-training has a substantial hidden capability cost. R2E-Gym alone reduces every non-agent and tool-use benchmark relative to the Instruct ceiling: LiveCodeBench by 13.10, BFCL by 7.40, FullStackBench-EN by 6.08, τ -bench by 2.30, and OJBench by 2.40. Averaged across the six benchmarks the post-trained model loses 4.81 points relative to Instruct—the implicit cost paid for SWE-Bench gains, rarely highlighted in agent papers.

Mid-training largely closes the regression gap. Adding FIM mid-training before the same post-training restores LiveCodeBench by +11.10, OJBench by +1.94 (within 0.46 of the Instruct ceiling), and FullStackBench-EN by +0.53. The six-benchmark average rises from 16.04 to 19.56 (+3.52 over post-training only), while the SWE-Bench-Verified gain on the same base is preserved (Table 1).

Table 3: Ablations on the 7B model with R2E-Gym post-training. (A) rationale source. (B) function-selection algorithm. (C) mask granularity (single vs. multi-function groups). All blocks share the baseline (*w/o mid-train*) and a controlled 200K-target budget; CoT is fixed to Gemini-3-Flash in (B) and (C). Bold marks the best configuration per block; the 80%/15%/5% mixture in (C) is the recipe used in our main results (Table 1), which is trained on the full corpus rather than the 200K budget here. Absolute numbers across all blocks therefore lie below the main-table recipe; the relative orderings within each block, rather than the absolute values, are the object of comparison.

Setting	SWE-Bench-Verified	SWE-Bench-Lite	Average
<i>(A) Chain-of-thought source (selection fixed to “full”)</i>			
w/o mid-train (baseline)	15.00	11.33	13.17
+ FIM, no CoT	16.10	12.60	14.35
+ FIM, self-CoT (Qwen2.5-Coder-7B-Instruct)	16.40	13.30	14.85
+ FIM, Gemini-3-Flash CoT	17.00	14.20	15.60
<i>(B) Function-selection algorithm (CoT fixed to Gemini-3-Flash)</i>			
w/o mid-train (baseline)	15.00	11.33	13.17
Random	15.30	12.60	13.95
Gemini-selected	16.40	13.70	15.05
PDG only	16.10	13.60	14.85
PDG + complexity ($\hat{H}$)	16.50	13.60	15.05
PDG + inferability ($\hat{I}$)	16.70	14.00	15.35
Full (PDG + $\hat{H}$ + $\hat{I}$)	17.00	14.20	15.60
<i>(C) Mask granularity (selection fixed to “full”, CoT fixed to Gemini-3-Flash)</i>			
w/o mid-train (baseline)	15.00	11.33	13.17
Single-function only	17.00	14.20	15.60
85% single + 15% pair ($k = 2$)	17.20	14.60	15.90
95% single + 5% triple ($k = 3$)	17.00	14.40	15.70
80% single + 15% pair + 5% triple	17.40	14.80	16.10

Mid-training therefore improves the cost–benefit profile of agentic post-training: in-domain target metrics improve and the bulk of off-distribution erosion is undone in the same training run.

Cross-domain transfer to non-coding tool use. τ -bench and BFCL contain no Python code-editing data, and our mid-training corpus contains no tool-use trajectories. Mid-training nevertheless improves τ -bench by +3.90 and BFCL by +2.40 over post-training alone, with a consistent recovery on Terminal-Bench 2.0 (+1.25). Because the corpus carries no tool-use signal, the only mechanism is a structural prior installed at mid-training that survives post-training, which is the direct evidence for the function-call/tool-call isomorphism (Section 2.1).

3.4 Ablation Studies

We run three controlled ablations on Qwen2.5-Coder-7B-Instruct with R2E-Gym post-training and a shared mid-training-free baseline. The first isolates the role of the chain-of-thought rationale; the second isolates the role of the function-aware selection pipeline; the third varies mask granularity (single-function vs. multi-function groups). Due to compute constraints, ablations run at 7B only.

FIM structure contributes independently of CoT distillation. Block (A) addresses the concern that gains stem mainly from distilling a frontier teacher. Removing the rationale entirely (*no CoT*) already lifts the average by +1.18—roughly half of the +2.43-point Gemini-3 gain—direct evidence that the function-aware FIM signal does substantive work before any reasoning supervision is added. Replacing Gemini-3 with rationales from the model under training (*self-CoT*) reaches 14.85, recovering 1.68 of the 2.43-point gap; the residual 0.75 points attributable to a frontier teacher are real but modest. The recipe is therefore not a thinly disguised distillation pipeline.

Function-selection algorithm is the dominant lever. Block (B) varies the selection algorithm with CoT and budget held fixed. *Random* masking sets a floor at 13.95, and *Gemini-selected* reaches 15.05: frontier judgment on *which* function to mask helps but is not sufficient. Restricting candidates to functions with at least one PDG neighbor (*PDG only*) reaches 14.85; adding $\hat{H}$ or $\hat{I}$ on top

yields 15.05 and 15.35—intrinsic difficulty and contextual recoverability each contribute beyond the structural filter. Combining them (*Full*) reaches 15.60, confirming that $\hat{H}$ and $\hat{I}$ are not redundant.

Mask granularity: pair masking helps, triples saturate. Block (C) mixes single-function targets with multi-function groups (Section 2.3.5). Adding 15% pair targets raises the average from 15.60 to 15.90; substituting 5% triple targets instead is essentially neutral (15.70). The full 80%/15%/5% mix achieves 16.10. This matches the analysis in Section 4.2: training on cross-function dependencies disproportionately helps tasks whose gold patches span multiple functions, but the marginal return from larger groups diminishes as coupling becomes harder to maintain under joint masking.

4 Analysis

The previous section established that FIM mid-training improves end-task metrics. We now ask *how* the resulting agent behaves differently and *where* along the trajectory the gains accrue. Both analyses use the 14B configuration with R2E-Gym, comparing the post-training-only baseline (*R2E-Gym*) against our recipe (*FIM-Midtrain + R2E-Gym*). The Lite analogue, the full action-type distribution, the no-patch mechanism, and a concrete trajectory contrast are deferred to Appendix D.

4.1 Recovery from Negative Observations

A trajectory contains a *negative observation* if any tool output matches a fixed set of error patterns (stack traces, “No replacement was performed”, shell errors, etc.; full list in Appendix B). The fraction of such trajectories is essentially identical across checkpoints (88.8% baseline vs. 91.8% ours), so the agents see comparable amounts of negative feedback. The *recovery rate*—the fraction of error-containing trajectories that nonetheless terminate with a passing patch—rises from 24.8% to 28.8% (+4.0 pp; +16% relative; Table 4). This is precisely the capability our framing predicts mid-training should support: continuing productively after an external return contradicts the model’s prior expectation. Mid-training also shifts the agent toward an iterate-and-verify policy, increasing edits per solved task from 3.3 to 7.4 and trajectory length from 15.1 to 23.6 steps; action-type breakdowns are in Appendix D.

Table 4: Headline trajectory metrics on SWE-Bench-Verified. Full metrics in Appendix D.

Setting	Recovery rate (%)	Edits / solved	Steps / solved	Pass (%)
+ R2E-Gym	24.8	3.3	15.1	26.2
+ FIM-Midtrain + R2E-Gym	28.8	7.4	23.6	29.2

4.2 The Gain Concentrates on Multi-Function Reasoning

The most direct test of the isomorphism argument (Section 2.1) is whether mid-training preferentially helps tasks that require reasoning about cross-function dependencies inside a file. We stratify the 500 Verified tasks by the shape of the gold patch (Figure 5, Appendix D). On the 88 tasks whose gold patch modifies ≥ 2 functions within a single file, the baseline solves 13.6% on average and ours solves 22.7%, an absolute gain of +9.1 pp—more than $4\times$ the gain on the 341 single-function tasks (+2.1 pp). Per-instance head-to-head on this bucket shows ours uniquely solves about twice as many tasks as the baseline. Multi-file tasks ($n=71$) are not differentially helped, which we attribute to a granularity mismatch: our FIM operates within files (Appendix D). The slice where mid-training helps most is precisely the slice where the agent must follow control- and data-dependencies between caller-callee or sibling functions inside a file—the same structure exposed by function-aware FIM masking. A complementary outcome-distribution breakdown (Figure 6) shows the +15-task gain is driven by an order-of-magnitude reduction in *no-patch* failures alongside small reductions in localization errors; the FIM signal conditions the model to produce a non-empty span between prefix and suffix—a disposition that survives subsequent post-training.

5 Related Work

Mid-training and continued pretraining. A growing body of work [36, 15] identifies a stage between pretraining and post-training in which a model is further trained on a curated corpus to install

inductive biases hard to acquire from generic web text or fine-tuning. MiniCPM [18], OLMo [13], and DeepSeek-V3 [25] schedule such a stage near the end of pretraining; Code-Llama [33] follows the same staging philosophy for context-length generalization. Recent work shows *when* specialized data is introduced matters as much as how much is used [19, 1]. We adopt this philosophy but target an *agent-oriented* structural prior through function-aware FIM.

Fill-in-the-middle and structure-aware code objectives. FIM [2, 8] is a defining ingredient of modern code LLM pretraining, used in Code-Llama [33], StarCoder-2 [27], CodeGen [30], Qwen-Coder [20], and DeepSeek-Coder [14]. A recent line moves beyond random spans toward *structure-aware* masking: AST-T5 [12] and AST-FIM [11] mask AST subtrees, the latter reporting up to +5 pts on real-edit FIM benchmarks; Horizon-Length Prediction [7] adds a planning signal; Instruction-aware FIM [34] extends the FIM tuple with a developer-comment slot. Repository-level retrieval methods such as GraphCoder [26] and DRACO [4] exploit program dependencies, but at inference time. Our recipe differs along three axes: targets are chosen at *function granularity* via PDG analysis with a complexity–inferability criterion, so the masked region is a unit of agent-relevant reasoning rather than a syntactic subtree; an explicit *chain-of-thought* sits inside the FIM middle; and the objective lives in a *dedicated mid-training stage*.

Coding agent foundation models. Progress has advanced on three fronts. Benchmarks have moved from SWE-Bench [23] to broader, contamination-resistant suites including Multi-SWE-Bench [46] and SWE-Bench-Pro [6]. Scaffolds such as SWE-agent [43], OpenHands [37], and Agentless [41] expose different action interfaces. Trajectory-centric post-training pipelines—SWE-Gym [31], R2E-Gym [22], SWE-Smith [44], SWE-Lego [35], and Skywork-SWE [47]—curate agent trajectories on top of these scaffolds, with recent extensions replacing or augmenting SFT with RL [10, 39]. We use R2E-Gym, SWE-Smith, and SWE-Lego unmodified atop a mid-trained checkpoint; the novelty lies one stage earlier—a self-supervised signal extracted from structure already present in source code, which mitigates the off-domain capability erosion trajectory-only post-training inflicts.

Distillation from frontier models. Our recipe uses Gemini-3-Flash to generate the CoT rationales embedded in each FIM middle, placing it within distillation from a strong teacher [29, 9, 40] and CoT distillation [16, 17, 3]. Recent work shows diversity and structural alignment of the rationale matter as much as teacher strength [3]; consistent with this, our CoT-source ablation (Section 3.4) confirms the recipe is not distillation-bound.

6 Limitations and Discussion

We close by stating four limitations that scope our claims. (i) *Python-only corpus and evaluation*. The mid-training corpus and the in-domain agent benchmarks are exclusively Python; cross-language evidence comes only indirectly through FullStackBench-EN (Section 3.3), and transfer to Java, C++, or Rust is left to future work. (ii) *Teacher dependency for CoT*. The default recipe relies on Gemini-3-Flash; the CoT-source ablation (Section 3.4) shows self-generated rationales recover most of the gain, but a fully open-source replication requires a comparably strong open teacher. (iii) *Partial cross-base validation*. Our cross-base evidence comes from a single non-Qwen2.5-Coder configuration (Qwen3-8B with SWE-Lego), which simultaneously varies the post-training pipeline; the result indicates the recipe is not tied to one pretraining/post-training combination rather than guaranteeing transfer across all base families. (iv) *Modularity assumption*. Function-aware FIM presupposes modular code; on monolithic scripts, generated code, or notebooks, the pipeline yields fewer eligible targets, a regime we do not study systematically.

7 Conclusion

A single step of a coding agent and a single function call site share the same four-part structure—*context, action, externally produced return, continuation*—making source code an internet-scale supply of agent-relevant signal. We turn this into function-aware FIM mid-training: a self-supervised stage that masks targets selected via program dependency graph analysis and a complexity–inferability double criterion, with chain-of-thought rationales embedded inside the FIM middle. Across model size, post-training pipeline, and base-model family, the recipe delivers consistent gains on coding-agent benchmarks, and the same Python-only corpus transfers to non-coding tool-use benchmarks (τ -bench, BFCL). Future work includes extending the selection to non-Python languages and composing mid-training with RL post-training.

References

- [1] Syeda Nahida Akter, Shrimai Prabhumoye, Eric Nyberg, Mostofa Patwary, Mohammad Shoeybi, Yejin Choi, and Bryan Catanzaro. Front-loading reasoning: The synergy between pretraining and post-training data. *arXiv preprint arXiv:2510.03264*, 2025.
- [2] Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle, 2022.
- [3] Xinghao Chen, Zhijing Sun, Guo Wenjin, Miaoran Zhang, Yanjun Chen, Yirong Sun, Hui Su, Yijie Pan, Dietrich Klakow, Wenjie Li, et al. Unveiling the key factors for distilling chain-of-thought reasoning. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 15094–15119, 2025.
- [4] Wei Cheng, Yuhan Wu, and Wei Hu. Dataflow-guided retrieval augmentation for repository-level code completion. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7957–7977, 2024.
- [5] Yao Cheng, Jianfeng Chen, Jie Chen, Li Chen, Liyu Chen, Wentao Chen, Zhengyu Chen, Shijie Geng, Aoyan Li, Bo Li, et al. Fullstack bench: Evaluating llms as full stack coders, 2024.
- [6] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, et al. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.
- [7] Yifeng Ding, Hantian Ding, Shiqi Wang, Qing Sun, Varun Kumar, and Zijian Wang. Horizon-length prediction: Advancing fill-in-the-middle capabilities for code generation with lookahead planning. 2024.
- [8] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. InCoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999*, 2022.
- [9] Sanchit Gandhi, Patrick Von Platen, and Alexander M Rush. Distil-whisper: Robust knowledge distillation via large-scale pseudo labelling, 2023.
- [10] Alexander Golubev, Maria Trofimova, Sergei Polezhaev, Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Sergey Abramov, Andrei Andriushchenko, Filipp Fisin, et al. Training long-context, multi-turn software engineering agents with reinforcement learning. *arXiv preprint arXiv:2508.03501*, 2025.
- [11] Linyuan Gong, Alvin Cheung, Mostafa Elhoushi, and Sida Wang. Structure-aware fill-in-the-middle pretraining for code. *arXiv preprint arXiv:2506.00204*, 2025.
- [12] Linyuan Gong, Mostafa Elhoushi, and Alvin Cheung. Ast-t5: Structure-aware pretraining for code generation and understanding. *arXiv preprint arXiv:2401.03003*, 2024.
- [13] Dirk Groeneveld, Iz Beltagy, Evan Walsh, Akshita Bhagia, Rodney Kinney, Oyvind Tafjord, Ananya Jha, Hamish Ivison, Ian Magnusson, Yizhong Wang, et al. Olmo: Accelerating the science of language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15789–15809, 2024.
- [14] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yifan Wu, YK Li, et al. DeepSeek-Coder: When the large language model meets programming – the rise of code intelligence, 2024.
- [15] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A Smith. Don’t stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pages 8342–8360, 2020.
- [16] Namgyu Ho, Laura Schmid, and Se-Young Yun. Large language models are reasoning teachers. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 14852–14882, 2023.

- [17] Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alex Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8003–8017, 2023.
- [18] Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. Minicpm: Unveiling the potential of small language models with scalable training strategies, 2024.
- [19] Junjie Huang, Jiarui Qin, Di Yin, Weiwen Liu, Yong Yu, Xing Sun, and Weinan Zhang. Remit: RL-guided mid-training for iterative llm evolution. *arXiv preprint arXiv:2602.03075*, 2026.
- [20] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2.5-Coder technical report, 2024.
- [21] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024.
- [22] Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural environments and hybrid verifiers for scaling open-weights SWE agents, 2025.
- [23] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The twelfth international conference on learning representations*, 2023.
- [24] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. StarCoder: May the source be with you!, 2023.
- [25] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [26] Wei Liu, Ailun Yu, Daoguang Zan, Bo Shen, Wei Zhang, Haiyan Zhao, Zhi Jin, and Qianxiang Wang. Graphcoder: Enhancing repository-level code completion via code context graph-based retrieval and language model. *arXiv preprint arXiv:2406.07003*, 2024.
- [27] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024.
- [28] Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026.
- [29] Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. Orca: Progressive learning from complex explanation traces of gpt-4, 2023.
- [30] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*, 2022.
- [31] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym, 2024.
- [32] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.
- [33] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. Code Llama: Open foundation models for code, 2023.

- [34] Zhensu Sun, Chengran Yang, Chao Peng, Pengfei Gao, Xiaoning Du, Li Li, and David Lo. Bridging developer instructions and code completion through instruction-aware fill-in-the-middle paradigm. *arXiv preprint arXiv:2509.24637*, 2025.
- [35] Chaofan Tao, Jierun Chen, Yuxin Jiang, Kaiqi Kou, Shaowei Wang, Ruoyu Wang, Xiaohui Li, Sidi Yang, Yiming Du, Jianbo Dai, et al. SWE-Lego: Pushing the limits of supervised fine-tuning for software issue resolving, 2026.
- [36] Chengying Tu, Xuemiao Zhang, Rongxiang Weng, Rumei Li, Chen Zhang, Yang Bai, Hongfei Yan, Jingang Wang, and Xunliang Cai. A survey on llm mid-training. *arXiv preprint arXiv:2510.23081*, 2025.
- [37] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. OpenHands: An open platform for AI software developers as generalist agents. 2024.
- [38] Zhexu Wang, Yiping Liu, Yejie Wang, Wenyang He, Bofei Gao, Muxi Diao, Yanxu Chen, Kelin Fu, Flood Sung, Zhilin Yang, et al. Ojbench: A competition level code benchmark for large language models, 2025.
- [39] Yuxiang Wei, Zhiqing Sun, Emily McMilin, Jonas Gehring, David Zhang, Gabriel Synnaeve, Daniel Fried, Lingming Zhang, and Sida Wang. Toward training superintelligent software agents through self-play swe-rl. *arXiv preprint arXiv:2512.18552*, 2025.
- [40] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. Magicoder: Empowering code generation with oss-instruct. *arXiv preprint arXiv:2312.02120*, 2023.
- [41] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.
- [42] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report, 2025.
- [43] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. volume 37, pages 50528–50652, 2024.
- [44] John Yang, Kilian Lieret, Carlos E Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents, 2025.
- [45] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.
- [46] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, et al. Multi-swe-bench: A multilingual benchmark for issue resolving. *arXiv preprint arXiv:2504.02605*, 2025.
- [47] Liang Zeng, Yongcong Li, Yuzhen Xiao, Changshi Li, Chris Yuhao Liu, Rui Yan, Tianwen Wei, Jujie He, Xuchen Song, Yang Liu, et al. Skywork-swe: Unveiling data scaling laws for software engineering in llms. *arXiv preprint arXiv:2506.19290*, 2025.

A Corpus Details

This appendix expands on the data-collection summary (Section 2.2). We report category coverage, per-property statistics, and the license inventory of the 968 released repositories.

A.1 License Inventory

Figure 4 shows the license distribution across the 968 released repositories. The corpus is dominated by permissive licenses (MIT, Apache-2.0, BSD-3); the remainder is split among copyleft (GPL, AGPL, LGPL, MPL) and Creative Commons families. Every license in the corpus permits at least non-commercial research use, and all small categories are aggregated in this chart as “Other research-permissive licenses.” The per-repository license is released alongside the repository list.

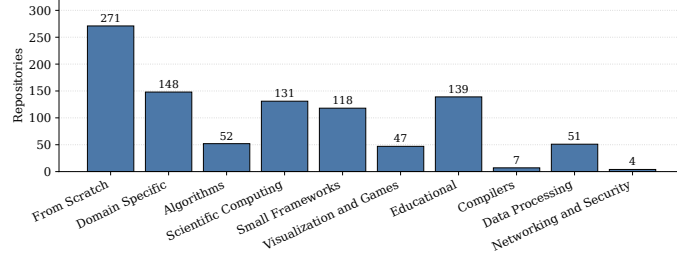


Figure 3: Distribution of the 968 source repositories across ten topic categories. The corpus is dominated by reference implementations, scientific computing, and small frameworks; compiler and networking/security tails are kept by design to maintain coverage diversity.

Table 5: Mid-training corpus statistics after decontamination and quality filtering. Token counts use the Qwen2.5-Coder tokenizer. All numeric quantities reflect the released corpus.

Property	Value
Source GitHub repositories	968
Topic categories	10
Filtered self-contained Python files	$\approx 78\text{K}$
Total FIM samples	$\approx 400\text{K}$
Mid-training token budget	$\approx 2.6\text{B}$
Single-function FIM targets	$\approx 320\text{K}$ ($\approx 2.0\text{B}$ tokens)
Multi-function targets ($k=2$)	$\approx 60\text{K}$ ($\approx 0.4\text{B}$ tokens)
Multi-function targets ($k=3$)	$\approx 20\text{K}$ ($\approx 0.2\text{B}$ tokens)
Mean target LoC	≈ 34
Targets with Gemini-3 CoT	100%
SWE-Bench source-repo overlap	0

B Algorithmic Details

This appendix collects the full algorithmic details summarized in Section 2.3. The complete Python implementation of both the single-function and multi-function selection pipelines, including all default hyperparameters listed below, is provided in the supplementary material.

B.1 Single-Function FIM Target Selection

Algorithm 1 summarizes the per-file pipeline that produces single-function FIM targets. It composes the program dependency graph (Section 2.3.1), the complexity score $\hat{H}$ (Section 2.3.2), the inferability score $\hat{I}$ (Section 2.3.3), the difficulty penalty ρ (Appendix B.5), and the hard filters described below.

We use $L_{\min} = 50$, $L_{\max} = 1800$, $\ell_{\min} = 10$, $\ell_{\max} = 200$, $\tau_{\hat{H}} = 0.15$, and $\tau_{\text{FIM}} = 0.08$ throughout.

B.2 Program Dependency Graph: Call Resolution

For every `Call` node inside the body of $u \in \mathcal{V}$, we attempt to resolve its callee to some $v \in \mathcal{V}$, adding a directed edge $u \rightarrow v$ in $\mathcal{E}_{\text{call}}$. Resolution handles three Python idioms:

- **Direct module-level invocations:** `foo(...)` is resolved by qualified-name lookup in the same module.
- **Class instantiation:** `ClassName(...)` is resolved to `ClassName.__init__`.
- **Method calls:** `self.m(...)` or `cls.m(...)` inside class bodies is resolved within the enclosing class scope.

When a call cannot be resolved exactly we fall back to short-name matching against the qualified-name index, which recovers most cross-module references while admitting a controlled false-positive

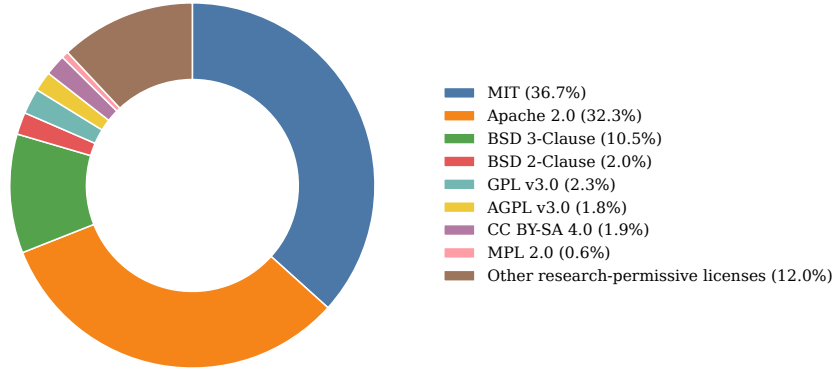


Figure 4: License distribution of the 968-repository corpus. Permissive licenses (MIT, Apache-2.0, BSD) account for over 80% of the corpus. Small categories (LGPL, ISC, Boost, MIT-0, Unlicense, CC0, CC BY, CC BY-NC, etc.) are aggregated as “Other research-permissive licenses,” all of which permit at least non-commercial research use.

Algorithm 1 Single-function FIM target selection for one file.

Require: Source file s ; thresholds $\tau_d, \tau_{\text{FIM}}, \tau_{\hat{H}}$; file-line bounds $[L_{\min}, L_{\max}]$; LoC bounds $[\ell_{\min}, \ell_{\max}]$

- 1: **if** $\text{Lines}(s) \notin [L_{\min}, L_{\max}]$ **then**
- 2: **return** $\emptyset$ ▷ file-level pre-filter
- 3: **end if**
- 4: $T \leftarrow \text{ParseAST}(s)$
- 5: $\mathcal{V}, \mathcal{E}_{\text{call}}, \mathcal{E}_{\text{sib}} \leftarrow \text{BuildPDG}(T)$ ▷ Section 2.3.1
- 6: $\mathcal{C} \leftarrow \emptyset$
- 7: **for each** $v \in \mathcal{V}$ **do**
- 8: **if** v is a dunder method **or** $\text{LoC}(v) \notin [\ell_{\min}, \ell_{\max}]$ **then**
- 9: **continue**
- 10: **end if**
- 11: Compute $\hat{H}(v)$ and $\hat{I}(v)$ ▷ Eqs. (1), (2)
- 12: **if** $\hat{H}(v) < \tau_{\hat{H}}$ **then continue**
- 13: **end if**
- 14: $\Delta(v) \leftarrow \max(0, \hat{H}(v) - \hat{I}(v)) / (\hat{H}(v) + \epsilon)$
- 15: $\text{FIM}(v) \leftarrow \frac{\hat{H}(v) \hat{I}(v)}{\hat{H}(v) + \hat{I}(v) + \epsilon} \cdot \rho(\Delta(v))$
- 16: **if** $\text{FIM}(v) \geq \tau_{\text{FIM}}$ **then**
- 17: $\mathcal{C} \leftarrow \mathcal{C} \cup \{v\}$
- 18: **end if**
- 19: **end for**
- 20: **return** $\mathcal{C}$ ▷ ranked by FIM descending

rate. Sibling edges $\mathcal{E}_{\text{sib}}$ are constructed by enumerating all pairs of methods belonging to the same class.

B.3 Complexity Score: Caps and Weights

The complexity score $\hat{H}(v)$ defined in Eq. (1) uses caps $(c_\ell, c_c, c_d) = (50, 10, 5)$ and weights $(w_\ell, w_c, w_d) = (0.4, 0.4, 0.2)$. The cap $c_\ell = 50$ on lines of code matches the median target length in our corpus; the cap $c_c = 10$ on McCabe complexity is roughly twice the median over our filtered functions; the cap $c_d = 5$ on nesting depth covers the tail of practical Python code. The cap value of

2 in $\phi(x, c) = \min(x/c, 2)$ allows genuinely complex functions to score above the median without letting outliers dominate.

B.4 Inferability Score: Component Formulas

The five components of $\hat{I}(v)$ (Eq. 2) are defined below. Default mixing weights are $(\alpha, \beta, \gamma, \delta, \varepsilon) = (0.30, 0.25, 0.20, 0.10, 0.15)$. All numerical constants used inside the components are listed in Table 6; the formulas reference them directly.

Caller signal C_{caller} . For each in-file caller u of v we compute a per-caller specificity $\text{sp}(u, v)$ by scanning every call site inside u that targets v (matched by short name). At each matching site we accumulate $b_{\text{site}} = 0.50$ for the call itself, plus $\eta_{\text{lit}} = 0.15$ per literal-constant argument, $\eta_{\text{name}} = 0.05$ per name argument, $\eta_{\text{other}} = 0.08$ per other expression argument, $\eta_{\text{kw}} = 0.12$ per keyword argument, and a $b_{\text{found}} = 0.10$ “found” bonus. The per-caller specificity is clipped at $\text{cap}_{\text{caller}} = 1.5$, with a fallback value of 0.5 when the caller does not actually invoke v by name. Summing over callers and normalizing, $C_{\text{caller}} = \min(\sum_u \text{sp}(u, v) / n_{\text{caller}}, 1)$, with normalization constant $n_{\text{caller}} = 3$.

Callee signal C_{callee} . $C_{\text{callee}} = \min(|\text{IntCallees}(v)| / n_{\text{callee}}, 1)$ where $\text{IntCallees}(v)$ is the set of intra-file functions that v calls, and $n_{\text{callee}} = 4$. A function that orchestrates known helpers is more recoverable than one calling only opaque externals.

Signature signal C_{sig} . A monotone aggregate of four sub-bonuses, capped at 1: $+0.30$ if a return-type annotation is present, $+0.25$ if any parameter carries a type annotation, $+\min(\text{nameparts}/5, 0.25)$ for the descriptiveness of the function name (split on `_`), and $+\min(|\text{params}_{\text{-self}}|/6, 0.20)$ for the count of non-`self`/`cls` parameters.

Documentation signal C_{doc} . $C_{\text{doc}} = 0.5$ if a docstring is present, otherwise 0. We do not score docstring informativeness, both for robustness and because Section 2.4 generates a richer textual rationale.

Class signal C_{class} . For methods, $C_{\text{class}} = \min(\text{sib}(v) / n_{\text{sib}}, 1)$ where $n_{\text{sib}} = 5$, plus a $b_{\text{init}} = 0.30$ bonus when a distinct `__init__` exists in the same class; the total is clipped at 1. Module-level functions receive $C_{\text{class}} = 0$.

Table 6: Numerical constants inside the $\hat{I}$ components.

Component	Constant	Value
C_{caller}	call-site base b_{site}	0.50
	literal-arg increment η_{lit}	0.15
	name-arg increment η_{name}	0.05
	other-arg increment η_{other}	0.08
	keyword-arg increment η_{kw}	0.12
	found bonus b_{found}	0.10
	per-caller cap $\text{cap}_{\text{caller}}$	1.5
	expected-callers norm n_{caller}	3
C_{callee}	expected-fan-out norm n_{callee}	4
C_{sig}	return-type bonus	$+0.30$
	param-type bonus	$+0.25$
	name-parts term	$\min(\text{parts}/5, 0.25)$
	non-self params term	$\min(p_{\text{-self}} /6, 0.20)$
C_{doc}	docstring present	0.5
C_{class}	sibling norm n_{sib}	5
	<code>__init__</code> bonus b_{init}	0.30

B.5 Difficulty Δ , Penalty ρ , and Hard Filters

The penalty $\rho(\Delta(v))$ in Eq. (3) is built from the residual difficulty $\Delta(v)$, defined as the share of complexity that is not explained by context:

$$\Delta(v) = \frac{\max(0, \hat{H}(v) - \hat{I}(v))}{\hat{H}(v) + \epsilon} \in [0, 1]. \quad (4)$$

The penalty ρ is one-sided: targets with $\Delta(v) \leq \tau_d$ are left intact, while targets above the threshold are damped by a Gaussian factor:

$$\rho(\Delta) = \begin{cases} 1, & \Delta \leq \tau_d, \\ \exp\left(-\frac{(\Delta - \tau_d)^2}{2\sigma^2}\right), & \Delta > \tau_d. \end{cases} \quad (5)$$

For single-function selection we use $\tau_d = 0.50$ and $\sigma = 0.20$. The asymmetry reflects an asymmetry in the underlying objective: trivially easy targets are already eliminated by the hard filters below, so we do not need to additionally reward high inferability; conversely, targets that remain hard even with full context become unlearnable noise and must be down-weighted.

Hard filters. Independent of the smooth score, we discard (i) files outside the line-count window $[L_{\min}, L_{\max}] = [50, 1800]$ entirely; (ii) functions with $\text{LoC}(v) \notin [10, 200]$ to control training-instance length; (iii) dunder methods (`__init__`, `__repr__`, etc.), which serve as scaffolding rather than reasoning targets; (iv) functions with $\hat{H}(v) < \tau_{\hat{H}} = 0.15$ to remove trivial cases; and (v) functions with $\text{FIM}(v) < \tau_{\text{FIM}} = 0.08$. A file with no surviving target contributes nothing to the corpus.

B.6 Worked Example: Scoring `Calculator.total`

To make the numbers in Figure 2(b) reproducible, this subsection walks through every quantity that goes into $\hat{H}$, $\hat{I}$, and the final FIM score for the running example. The target function is:

```
def total(self) -> int:
    """Sum positive integer entries."""
    s, n = 0, 0
    for v in self.history:
        if is_int(v):
            if v > 0:
                s = add(s, v)
                n += 1
    if n == 0: return 0
    return s
```

Lines of code (LoC = 10). Following Python’s ast convention, $\text{LoC}(v) = \text{end_lineno} - (\text{lineno} - 1)$, the number of source lines from the `def` keyword through the last statement, both inclusive. Counting the ten lines above (including the docstring) gives $\text{LoC} = 10$.

Cyclomatic complexity (CC = 5). The McCabe counter starts at 1 and adds 1 per branching node: `If/IfExp`, `For/While`, `ExceptionHandler`, (values−1) per `BoolOp`, and one per generator inside a comprehension. For `Calculator.total`: base (+1), the `for` loop (+1), `if is_int` (+1), `if v>0` (+1), `if n==0` (+1), totalling $\text{CC} = 5$. Tuple unpacking and `n+=1` are not branches.

Maximum nesting depth (D = 3). Depth increments only on structural containers (`If`, `For`, `While`, `AsyncFor`, `With`, `AsyncWith`, `Try`). The deepest path in `total` is `for` → `if is_int` → `if v>0`, giving $D = 3$.

Complexity score $\hat{H} = 0.40$. Plugging into Eq. 1 with the default $(w_\ell, w_c, w_d) = (0.4, 0.4, 0.2)$ and $(c_\ell, c_c, c_d) = (50, 10, 5)$:

$$\begin{aligned} w_\ell \phi(10, 50) &= 0.4 \cdot 0.20 = 0.08, & w_c \phi(5, 10) &= 0.4 \cdot 0.50 = 0.20, \\ w_d \phi(3, 5) &= 0.2 \cdot 0.60 = 0.12, & \hat{H}(\text{total}) &= 0.08 + 0.20 + 0.12 = 0.40. \end{aligned}$$

Inferability score $\hat{I} = 0.48$. (i) *Caller specificity*. The only in-file caller is `Calculator.mean`, which invokes `self.total()` with no arguments and no keywords. Per Table 6, $sp = b_{\text{site}} + b_{\text{found}} = 0.50 + 0.10 = 0.60$. Aggregating and normalizing by $n_{\text{caller}} = 3$, $C_{\text{caller}} = \min(0.60/3, 1.0) = 0.20$ and $\alpha C_{\text{caller}} = 0.06$. (ii) *Callee count*. `total` calls `is_int` and `add` in-file (`self.history.append(v)` does not count because `append` is not in $\mathcal{V}$), so $C_{\text{callee}} = \min(2/n_{\text{callee}}, 1.0) = \min(2/4, 1.0) = 0.50$ and $\beta C_{\text{callee}} = 0.13$. (iii) *Signature*. Sub-bonuses $+0.30$ for `-> int`; $+ \min(1/5, 0.25) = 0.20$ for the descriptive name `total`; 0 for parameter type annotations; 0 for non-`self` parameters: $C_{\text{sig}} = 0.50$ and $\gamma C_{\text{sig}} = 0.10$. (iv) *Documentation*. A docstring is present, so $C_{\text{doc}} = 0.5$ and $\delta C_{\text{doc}} = 0.05$. (v) *Class context*. `total` sits among three siblings; the base score is $\min(3/n_{\text{sib}}, 1.0) = \min(3/5, 1.0) = 0.60$, plus the $b_{\text{init}} = 0.30$ bonus for the distinct `__init__` of `Calculator`, giving $C_{\text{class}} = 0.90$ and $\varepsilon C_{\text{class}} = 0.14$. Summing: $\hat{I}(\text{total}) = 0.06 + 0.13 + 0.10 + 0.05 + 0.14 = 0.48$.

Final FIM score and selection. $\Delta(\text{total}) = \max(0, -0.08)/0.40 = 0$, so $\rho(\Delta) = 1$ and $\text{FIM}(\text{total}) = (0.40 \cdot 0.48)/(0.40 + 0.48) \approx 0.22 \geq \tau_{\text{FIM}} = 0.08$, selecting it as a mask target. The four short helpers are removed before scoring by the LoC hard filter; `Calculator.__init__` is removed by the dunder filter.

B.7 Multi-Function Group Score

Group score. For a group $G = \{v_1, \dots, v_k\}$ enumerated from one of eight topology patterns,

$$\text{FIM}(G) = \text{Coup}(G) \cdot \frac{\hat{H}(G) \hat{I}(G)}{\hat{H}(G) + \hat{I}(G) + \epsilon} \cdot \rho^G(\Delta(G)), \quad (6)$$

where $\hat{H}(G) = \frac{1}{k} \sum_v \hat{H}(v)$ averages individual complexities, $\Delta(G)$ is defined analogously to the single-function case, ρ^G uses group-specific parameters $\tau_d^G = 0.55$ and $\sigma^G = 0.20$ (slightly more permissive than the single-function $\tau_d = 0.50$, reflecting that joint masking inherently lowers $\hat{I}$), and the coupling term $\text{Coup}(G) \in [0, 1]$ is a normalized count of intra-group edges plus a shared-state ratio:

$$\text{Coup}(G) = w_c \frac{|E_G^{\text{call}}|}{k(k-1)} + w_s \frac{|E_G^{\text{sib}}|}{\binom{k}{2}} + w_{\text{st}} \overline{\text{Jacc}}_G, \quad (7)$$

with $\overline{\text{Jacc}}_G$ the mean Jaccard similarity over pairs in G between sets of accessed instance attributes (`self.x`). We use $(w_c, w_s, w_{\text{st}}) = (0.50, 0.20, 0.30)$. The coupling term enters multiplicatively because, unlike the single-function case, the value of a multi-function target derives specifically from cross-function dependencies; a weakly coupled group is indistinguishable from k independent samples.

Group inferability. $\hat{I}(G)$ is recomputed under the assumption that *all* members of G are masked simultaneously: caller, callee, and sibling contributions from inside G are excluded; docstrings of group members contribute zero (they are part of the masked body); and the `__init__` bonus in C_{class} is awarded only when `__init__` is itself outside G . Signature information is retained because we do not mask function signatures. The remaining constants ($b_{\text{site}}, \eta_{\text{lit}}, \dots$) in Table 6 are unchanged. This re-computation prevents groups from receiving spurious credit for intra-group references that disappear under joint masking.

Topology taxonomy. A candidate group must form a connected subgraph in $\mathcal{E}_{\text{call}} \cup \mathcal{E}_{\text{sib}}$. We enumerate eight patterns:

- $k = 2$: *caller-callee* (direct $A \rightarrow B$ edge); *co-callee* (two callees of a common in-file caller); *sibling-coupled* (same-class methods sharing ≥ 1 instance attribute); *mutual-call* ($A \rightleftharpoons B$).
- $k = 3$: *call-chain* ($A \rightarrow B \rightarrow C$); *hub* ($A \rightarrow B, A \rightarrow C$); *fan-in* ($B \rightarrow A, C \rightarrow A$); *class-triad* (three pairwise state-sharing methods of the same class).

Group selection. We score every candidate group emitted by the topology enumerator and discard those failing any of the following defaults: each member must satisfy $\text{LoC} \in [10, 200]$ and not be a dunder; $\text{Coup}(G) \geq \tau_{\text{coup}}^G = 0.15$; $\hat{H}(G) \geq \tau_{\hat{H}}^G = 0.15$; total LoC ratio of the group relative to the file does not exceed $\theta_2 = 0.30$ for $k = 2$ or $\theta_3 = 0.40$ for $k = 3$; and the score clears a size-specific floor, $\text{FIM}(G) \geq \tau_{\text{FIM},2}^G = 0.04$ for pairs and $\text{FIM}(G) \geq \tau_{\text{FIM},3}^G = 0.03$ for triples. Remaining

candidates are sorted by $\text{FIM}(G)$ and selected greedily under non-overlap: a function may belong to at most one selected group per file. Per-file caps $(N_2, N_3) = (5, 3)$ prevent any single file from dominating the corpus.

Algorithm 2 Multi-function group FIM target selection for one file.

Require: Source file s ; per-function scores $\hat{H}(v), \hat{I}(v)$; thresholds $\tau_{\text{coup}}^G = 0.15$, $\tau_{\hat{H}}^G = 0.15$, $\tau_{\text{FIM},2}^G = 0.04$, $\tau_{\text{FIM},3}^G = 0.03$; LoC ratio caps $(\theta_2, \theta_3) = (0.30, 0.40)$; per-file caps $(N_2, N_3) = (5, 3)$

- 1: $\mathcal{V}, \mathcal{E}_{\text{call}}, \mathcal{E}_{\text{sib}} \leftarrow \text{BuildPDG}(\text{ParseAST}(s))$
- 2: $\mathcal{G} \leftarrow \emptyset$
- 3: **for** each topology pattern $P \in \{\text{caller-callee, co-callee, sibling-coupled, mutual-call, call-chain, hub, fan-in, class-triad}\}$ **do**
- 4: Enumerate all connected subgraphs $G \subseteq \mathcal{V}$ matching P in $\mathcal{E}_{\text{call}} \cup \mathcal{E}_{\text{sib}}$
- 5: **for** each candidate G **do**
- 6: **if** any $v \in G$ fails the per-function filter **or** $\text{LoC_ratio}(G, s) > \theta_{|G|}$ **or** $\text{Coup}(G) < \tau_{\text{coup}}^G$ **or** $\hat{H}(G) < \tau_{\hat{H}}^G$ **then**
- 7: **continue**
- 8: **end if**
- 9: Recompute $\hat{I}(G)$ under joint masking ▷ exclude intra-group references; $C_{\text{doc}} = 0$
- 10: $\Delta(G) \leftarrow \max(0, \hat{H}(G) - \hat{I}(G)) / (\hat{H}(G) + \epsilon)$
- 11: $\text{FIM}(G) \leftarrow \text{Coup}(G) \cdot \frac{\hat{H}(G) \hat{I}(G)}{\hat{H}(G) + \hat{I}(G) + \epsilon} \cdot \rho^G(\Delta(G))$
- 12: **if** $\text{FIM}(G) \geq \tau_{\text{FIM},|G|}^G$ **then**
- 13: $\mathcal{G} \leftarrow \mathcal{G} \cup \{(G, P, \text{FIM}(G))\}$
- 14: **end if**
- 15: **end for**
- 16: **end for**
- 17: Sort $\mathcal{G}$ by $\text{FIM}(G)$ descending
- 18: $\mathcal{S} \leftarrow \emptyset$
- 19: **for** each $(G, P, \text{FIM}(G)) \in \mathcal{G}$ **do**
- 20: **if** any $v \in G$ already covered by $\mathcal{S}$ **or** per-size cap (N_2 for $k=2$, N_3 for $k=3$) exceeded **then**
- 21: **continue**
- 22: **end if**
- 23: $\mathcal{S} \leftarrow \mathcal{S} \cup \{G\}$
- 24: **end for**
- 25: **return** $\mathcal{S}$

B.8 Worked Example: A caller-callee Pair

To illustrate the group score, consider a small file with two top-level functions:

```
def normalize(text: str) -> str:
    """Lowercase and strip punctuation."""
    out = []
    for ch in text:
        if ch.isalnum() or ch.isspace():
            out.append(ch.lower())
    return "".join(out)

def word_freq(doc: str) -> dict[str, int]:
    """Count tokens of 'doc' after normalization."""
    cleaned = normalize(doc)
    counts = {}
    for tok in cleaned.split():
        counts[tok] = counts.get(tok, 0) + 1
    return counts
```

Topology and pre-checks. The pair $G = \{\text{normalize}, \text{word_freq}\}$ matches the *caller-callee* pattern with one call edge $\text{word_freq} \rightarrow \text{normalize}$. They are top-level (no class), so the sibling component of Coup is zero and the shared-state component is zero (no `self._` accesses). The combined LoC is roughly 14 lines, well under the $\theta_2 = 0.30$ file-ratio cap.

Coupling. With one call edge in a pair, $|E_G^{\text{call}}|/(k(k-1)) = 1/2$. Plugging into Eq. 7 with $(w_c, w_s, w_{st}) = (0.50, 0.20, 0.30)$: $\text{Coup}(G) = 0.50 \cdot 0.5 + 0 + 0 = 0.25$, comfortably above the $\tau_{\text{coup}}^G = 0.15$ floor.

Group $\hat{H}$. Per-function complexities (computed as in Section 2.3.2) come out to roughly $\hat{H}(\text{normalize}) \approx 0.36$ and $\hat{H}(\text{word_freq}) \approx 0.30$, giving $\hat{H}(G) \approx 0.33$.

Group $\hat{I}$ under joint masking. Both functions are masked, so the call edge between them no longer contributes to C_{caller} or C_{callee} . Signature information is retained: both have descriptive names and full type annotations (strong C_{sig}); docstrings vanish under joint masking ($C_{\text{doc}} = 0$). With no surviving caller/callee/class signal, the group inferability collapses to $\hat{I}(G) \approx 0.18$, dominated by the signature term.

Final group FIM. The harmonic-mean-like product $\hat{H}\hat{I}/(\hat{H} + \hat{I}) \approx (0.33 \cdot 0.18)/0.51 \approx 0.116$; $\Delta(G) = (0.33 - 0.18)/0.33 \approx 0.45 < \tau_d^G = 0.55$ so $\rho^G = 1$; and $\text{Coup}(G) = 0.25$: $\text{FIM}(G) \approx 0.25 \times 0.116 \approx 0.029$. This example sits below the pair floor $\tau_{\text{FIM},2}^G = 0.04$ and would therefore *not* be selected—the toy module is too small for the joint-masking inferability to remain large enough. Real corpus pairs that pass the threshold typically come from class-method pairs with non-trivial sibling and shared-state coupling, where $\hat{I}(G)$ retains substantial C_{class} contribution even under joint masking. The walkthrough nonetheless shows the mechanism: joint masking strictly reduces $\hat{I}$ relative to the single-function average, and the coupling factor must compensate.

B.9 Chain-of-Thought Generation Prompt Template

Each FIM rationale–implementation pair is produced by Gemini-3-Flash in a single forward pass: given a Python source file with one function body redacted, the model is asked to first reason about what the body should do and then write a candidate implementation. Crucially, the model is *not* shown the ground-truth body at this stage; both the rationale and the predicted body must be derived from the surrounding context alone. The (rationale, predicted body) pairs that survive the quality filter (Appendix B.10) are then used to assemble mid-training samples (Appendix B.11).

[System] You are an expert Python programmer.

[User] Below is a Python file where one function’s body has been replaced with #<MASKED_FUNCTION_BODY>. Complete the masked function from the surrounding context.

Procedure. Analyze the visible context—imports, sibling functions, classes, call sites, and helpers invoked by the target—then reason step by step about what the function must do given its signature, type hints, and usage pattern. Only after this analysis, write an implementation consistent with the file’s coding style.

Output format. Return two sections in order: `### Reasoning` (a step-by-step natural-language analysis) and `### Implementation` (a fenced python block containing the function body only, without the signature).

<file with target body redacted>

The function to complete: <function name>.

For brevity, the skeleton condenses three pieces of the production prompt that govern output formatting rather than the task itself: (i) the explicit three-step task breakdown (analyze context, reason, implement) and the four sub-bullets under the reasoning step (signature semantics, intra-file usage, callee dependencies, overall purpose), which we collapse into a single *Procedure* paragraph and which direct the model toward the same context sources scored by $\hat{I}$ (Section 2.3.3); (ii) the formatting checklist enforcing body-only output, four-space indentation, and explicit docstring handling; and (iii) the verbatim Markdown headers and code-fence delimiters that make the two sections programmatically separable. The condensed material specifies *how* the response is formatted but does not change *what* is being elicited. We sample with the Gemini API’s near-greedy configuration

(temperature = 0, topK = 1); the verbatim prompt and all post-processing logic will be released with the corpus to support exact replication.

B.10 Completion-Quality Filtering Prompt Template

Each candidate FIM sample, paired with a trial completion, is screened by an LLM judge that assesses both whether the masked function is recoverable from its surrounding context and the quality of the completion against the ground truth. Filtering is performed by Gemini-3-Flash using the following prompt skeleton:

```
[System] You are an expert code reviewer.
[User] A model was asked to complete a masked function from its surrounding code context.
Evaluate the result along two axes: (i) whether the function is feasible to complete from
context alone, and (ii) the quality of the completion compared with the ground truth.
The source file with the target body redacted (shown as <MASKED>):
<file with target body redacted>
The function name, the ground-truth body, and the model's completion:
<function name>, <ground-truth body>, <model completion>
Part 1 – Feasibility. Mark the function infeasible if it depends on uncommon external APIs,
external conventions, or magic constants that cannot be inferred from the visible context;
otherwise mark it feasible.
Part 2 – Quality. Score the completion on a 1–5 scale along five dimensions—correctness,
executability, API usage, readability, and completeness—each with a one-sentence justification,
then assign an overall 1–5 score.
Output. Return a JSON object containing the feasibility verdict, the five component scores
with reasons, the overall score, and a should_discard flag set to true when the function
is infeasible, when executability = 1, or when both overall_score ≤ 2 and
executability ≤ 2.
```

For brevity, the skeleton above omits three pieces of the production prompt that anchor the scores rather than alter the procedure: (i) the per-level rubric for each of the five dimensions (e.g., *correctness* 5 = “functionally identical,” 3 = “partially correct, some important cases wrong,” 1 = “completely incorrect”), which fixes a consistent scale across samples; (ii) the expanded feasibility checklist enumerating concrete infeasibility factors (reliance on niche libraries, knowledge of external conventions, insufficient context, system-specific magic constants) and corresponding feasibility indicators; and (iii) the full JSON schema with named fields for every component reason, key similarities/differences, and a free-text `discard_reason`. The omitted material specifies *how* each axis is graded but does not change *what* is graded; We sample with the Gemini API’s near-greedy configuration (temperature = 0, topK = 1). The verbatim prompt together with all post-processing logic will be released with the corpus to support exact replication.

B.11 Mid-Training Sample Formatting Template

After upstream CoT generation (Appendix B.9) and quality filtering (Appendix B.10), each retained sample is serialized into a chat-style turn pair: the user turn poses the FIM completion task on the file with one function body redacted, and the assistant turn carries Gemini’s rationale followed by Gemini’s predicted body, both produced in the upstream pass. The ground-truth body from the source repository is used only by the filter to admit the sample; it does not appear in the training target.

```
[System] You are an expert Python programmer.
[User] Below is a Python file where one function's body has been replaced with #
<MASKED_FUNCTION_BODY>. Complete the masked function from the surrounding context.
[Procedure and output-format instructions identical to Appendix B.9.]
<file with target body redacted>
The function to complete: <function name>.
[Assistant]
### Reasoning
<Gemini-produced rationale>
### Implementation
```python
```

<Gemini-produced implementation>  
 ““

## B.12 Negative-Observation Patterns

The recovery-rate analysis (Section 4.1) flags a tool output as a *negative observation* when it matches any of the following patterns: Python stack-trace prefixes (Traceback (most recent call last):); error class names (SyntaxError, IndentationError, NameError, ImportError, AttributeError, TypeError, ValueError); shell error markers (command not found, permission denied, No such file or directory); and harness-specific failure strings (No replacement was performed, Patch did not apply, tests failed). Patterns are matched case-insensitively.

## C Training Hyperparameters

We use LLAMAFACTORY for FIM mid-training, R2E-Gym post-training, and SWE-Lego post-training, and TORCHTUNE for SWE-Smith post-training. All runs use AdamW with bf16 mixed precision and a cosine learning-rate schedule. For R2E-Gym and SWE-Smith we follow the official release scripts; for SWE-Lego we use 2 epochs instead of the official 4 to prevent overfitting (all other hyperparameters unchanged). Tables 7 and 8 list the exact settings; effective (global) batch sizes assume 8 GPUs.

Table 7: FIM mid-training hyperparameters, applied uniformly to all three base models (Qwen2.5-Coder-7B-Instruct, Qwen2.5-Coder-14B-Instruct, and Qwen3-8B).

Hyperparameter	Value
Optimizer	AdamW
Learning rate	$1.0 \times 10^{-5}$
LR schedule	Cosine
Warmup ratio	0.1
Weight decay	0.05
Epochs	1
Per-device batch size	1
Gradient accumulation	16
Effective batch size	128
Sequence length	32,768
Precision	bf16

Table 8: Agentic post-training hyperparameters for the three pipelines. R2E-Gym and SWE-Smith follow their official released scripts; SWE-Lego follows the official recipe except for the epoch count.

Hyperparameter	R2E-Gym	SWE-Smith	SWE-Lego
Base model	Qwen2.5-Coder-7B-Inst.	Qwen2.5-Coder-7B-Inst.	Qwen3-8B (FIM-midtrained)
Optimizer	AdamW	AdamW (fused)	AdamW
Learning rate	$1.0 \times 10^{-5}$	$1.0 \times 10^{-4}$	$1.0 \times 10^{-4}$
LR schedule	Cosine	Cosine	Cosine
Warmup	ratio 0.05	5 steps	ratio 0.1
Weight decay	0.0	0.01	0.01
Epochs	2	3	2 <sup>†</sup>
Per-device batch size	1	1	1
Gradient accumulation	1	4	8
Effective batch size	8	32	64
Sequence length	32,768	32,768	40,960
Precision	bf16	bf16	bf16

<sup>†</sup> Deviates from the official SWE-Lego recipe (4 epochs); reduced to 2 to prevent overfitting on the FIM-midtrained Qwen3-8B base.

### C.1 Compute Resources

All experiments are run on a single node of 8 NVIDIA H100 80 GB GPUs. Reproducing the full set of results in this paper—FIM mid-training of three base models (Qwen2.5-Coder-7B/14B-Instruct and Qwen3-8B), the three agentic post-training pipelines (R2E-Gym, SWE-Smith, SWE-Lego) and their corresponding mid-trained variants, the data/CoT/granularity ablations of Section 3.4, and the multi-seed evaluation sweeps—takes roughly 30 days of wall-clock time on the 8-GPU node, i.e.  $\approx 5,760$  GPU-hours in total. Preliminary and discarded runs not reported in this paper account for an additional  $\sim 30\%$  of compute.

## D Extended Behavioral Analysis (SWE-Bench-Verified)

This appendix collects the trajectory-level material that supplements the headline analysis in Section 4. Unless stated otherwise, statistics are run-means over three independent evaluation runs of each checkpoint on SWE-Bench-Verified (500 instances per run).

### D.1 Pass Rate by Gold-Patch Shape

Figure 5 reports the pass-rate stratification used in Section 4.2: the largest gain (+9.1 pp) is on multi-function single-file tasks ( $n=88$ ), more than  $4\times$  the +2.1 pp gain on single-function tasks ( $n=341$ ); multi-file tasks ( $n=71$ ) remain hard for both checkpoints ( $\sim 11.3\%$  each).

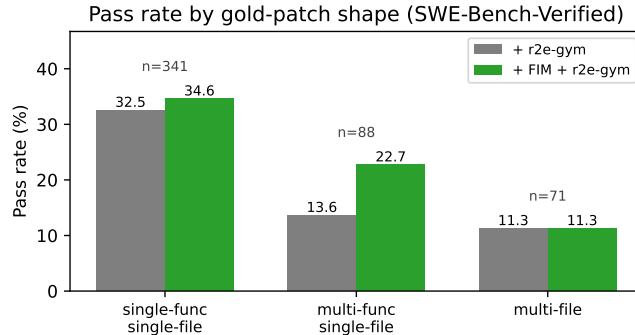


Figure 5: Pass rate on SWE-Bench-Verified (14B + R2E-Gym) stratified by gold-patch shape, run-means over three evaluation runs per checkpoint.

### D.2 Full Trajectory Metrics

Table 9 reports the full trajectory metrics summarized in Table 4, including unsolved-task breakdowns and output/context summaries.

### D.3 Iterate-and-Verify: Action-Type Distribution

Mid-training shifts the action-type distribution toward an iterate-and-verify policy. The share of `search` actions falls from 15.3% to 11.0%, while the share of `execute_bash` actions rises from 19.5% to 24.6%: the agent spends a smaller fraction of its budget on passive repository lookup and a larger fraction on actively running scripts and tests, then refining its edit. The cost is more steps—ours hits the step cap on  $\sim 45\%$  of trajectories vs. 7% for the baseline—but this cost is paid on the unsolved tail; on the solved set the additional steps translate into more correct patches.

### D.4 Failure-Mode Breakdown

We label every failed trajectory using a deterministic patch-quality cascade: *no-patch* (output diff is empty); *localization error* (non-empty patch but zero file overlap with the gold patch); *patch error* (file overlap with gold patch but tests fail). Figure 6 reports the outcome distribution: across the three runs, mid-training cuts no-patch failures by an order of magnitude ( $\sim 11 \rightarrow \sim 1$  trajectories per run),



Table 9: Trajectory-level behavioral metrics on SWE-Bench-Verified (14B, R2E-Gym), run-means over three evaluation runs of each checkpoint. Steps, edits, and errors-encountered are means per task. “Empty patch” is the fraction of *failed* trajectories whose final output\_patch is empty; “Loc. correct” is the fraction of all trajectories that locate at least one file overlapping the gold patch; “Ctx @ success” is the mean prompt context length on solved trajectories.

Setting	Steps / task		Edits / task		Errors enc.		Recovery rate (%)
	solved	unsolved	solved	unsolved	solved	unsolved	
+ R2E-Gym	15.1	22.4	3.3	5.4	3.5	5.8	24.8
<b>+ FIM-Midtrain + R2E-Gym</b>	<b>23.6</b>	<b>32.7</b>	<b>7.4</b>	<b>10.9</b>	<b>6.2</b>	<b>8.1</b>	<b>28.8</b>
<i>Output and context summaries</i>							
	Empty patch (% failed)		Loc. correct (% all)		Ctx @ success (tok)		Pass (%)
+ R2E-Gym	3.0		70.6		11,826		26.2
<b>+ FIM-Midtrain + R2E-Gym</b>	<b>0.3</b>		<b>73.4</b>		<b>16,397</b>		<b>29.2</b>

trims localization errors slightly ( $\sim 131 \rightarrow \sim 126$ ), and leaves the patch-error count essentially flat ( $\sim 227 \rightarrow \sim 227$ ). The +15-task gain in solved count is therefore primarily driven by the no-patch mode collapse.

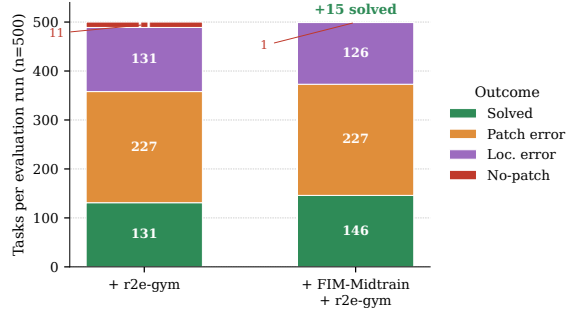


Figure 6: Outcome distribution per evaluation run on SWE-Bench-Verified (14B, R2E-Gym), averaged over three runs.

## D.5 No-Patch Mode: Mechanism

The baseline no-patch failures recovered by mid-training are distributed across all task buckets and are not concentrated in any single repository. In every recovered case the baseline emitted `<finish>` without applying any `str_replace`, while ours applied at least one edit. We interpret this as a direct consequence of the FIM training signal: under FIM the model is always conditioned to produce a non-empty token span between the prefix and suffix, and this disposition survives the post-training pipeline.

## D.6 Multi-File Tasks Are Not Differentially Helped

On the 71 Verified tasks whose gold patch spans  $\geq 2$  files, both checkpoints solve  $\sim 11.3\%$  on average and per-instance head-to-head on this bucket is balanced. We attribute this to a granularity mismatch: our function-aware FIM mid-training operates at the function level within files, so cross-file coordination is not directly trained for. Extending the selection algorithm to cross-file function pairs is a natural follow-up.

## D.7 A Concrete Contrast

The shift in termination behavior is clearest on cases where the baseline quits prematurely. On `scikit-learn-26323`, for instance, the baseline trajectory runs for a single step in every run, immediately emits `<finish>` with an empty edit and the wrong target file, and is scored as a failure;

on the same task our agent runs  $\sim 41$  steps, applies on the order of 20 `str_replace` operations, observes multiple negative tool returns, and recovers to a passing patch. Pooling across the three runs, on the order of  $\sim 16$  of the  $\sim 55$  tasks that ours uniquely solves on Verified have this signature: the baseline emits `<finish>` with zero file overlap with the gold patch (often before any negative observation has even arrived), while ours iterates past intermediate signals and converges on a correct edit. Mid-training does not enable a categorically new capability on these instances; it changes the agent’s stopping policy.

## E Behavioral Analysis on SWE-Bench-Lite

This appendix replicates the Verified analysis on the SWE-Bench-Lite test split (300 instances). The same trajectory-parsing pipeline and failure-mode taxonomy are used; statistics are run-means over three independent evaluation runs of each checkpoint.

The qualitative trends from Verified hold on Lite: mid-training raises the recovery rate from 18.8% to 22.1%, eliminates the no-patch failure mode entirely (baseline averages  $\sim 8$  no-patch failures per run, ours averages 0), and increases edit iteration on solved trajectories from 4.4 to 7.8 `str_replace` operations per task. The *multi-function* concentration of gain documented on Verified is not visible on Lite because Lite contains no multi-file tasks and only 54 multi-function single-file tasks; the bulk of Lite is single-function single-file tasks, on which the gain is +4.9 pp (19.5%  $\rightarrow$  24.4%).

Table 10: Trajectory-level behavioral metrics on SWE-Bench-Lite (14B, R2E-Gym), run-means over three evaluation runs of each checkpoint. Definitions match Table 9.

Setting	Steps / task		Edits / task		Errors enc.		Recovery rate (%)
	solved	unsolved	solved	unsolved	solved	unsolved	
+ R2E-Gym	17.5	21.3	4.4	5.2	4.8	5.4	18.8
<b>+ FIM-Midtrain + R2E-Gym</b>	<b>24.6</b>	<b>31.7</b>	<b>7.8</b>	<b>11.1</b>	<b>5.7</b>	<b>7.5</b>	<b>22.1</b>
<i>Output and context summaries</i>							
	Empty patch (% failed)		Loc. correct (% all)		Ctx @ success (tok)		Pass (%)
+ R2E-Gym	3.3		65.0		13,958		18.0
<b>+ FIM-Midtrain + R2E-Gym</b>	<b>0.0</b>		<b>71.0</b>		<b>18,104</b>		<b>22.0</b>

Failure-mode counts on Lite move in the same direction as on Verified: the no-patch mode is eliminated ( $\sim 8 \rightarrow 0$  trajectories per run), localization errors fall by about 11, and patch errors rise by about 7, reflecting the same iterate-and-verify shift.

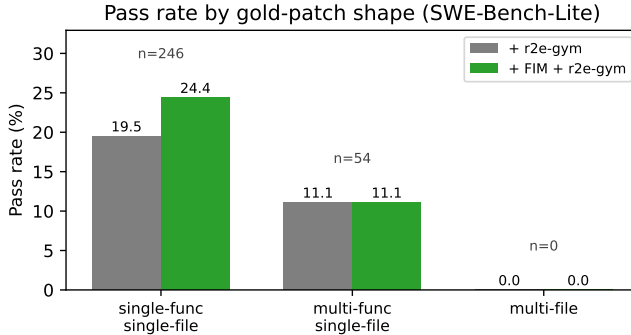


Figure 7: Pass rate on SWE-Bench-Lite stratified by gold-patch shape, averaged over three evaluation runs per checkpoint. Lite contains no multi-file tasks; the multi-function single-file bucket is small ( $n=54$ ) and shows no gain on this slice, with the +4.0 pp end-task improvement coming entirely from the single-function single-file bucket ( $n=246$ , +4.9 pp).

**Per-instance head-to-head.** Aggregating per-task outcomes across the three runs (run-majority vote per checkpoint), the differential favors ours on Lite by a  $\sim 1.8\times$  ratio overall and by a  $\sim 2.1\times$  ratio

on the single-function single-file bucket. On the multi-function single-file bucket the differential is balanced, consistent with the small absolute number of such tasks in Lite.

## NeurIPS Paper Checklist

### 1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 state the function-call/agent-step correspondence, the function-aware FIM mid-training method, and the three-axis robustness claim, each of which is supported by the experiments in Section 3.2 (Table 1) and Section 3.3 (Table 2).

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

### 2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 6 explicitly enumerates four limitations: Python-only corpus and evaluation, teacher dependency for CoT generation, partial cross-base validation (only one non-Qwen2.5-Coder configuration), and the modularity assumption underlying function-aware selection.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

### 3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper is empirical and contains no theorems or formal theoretical results; the function-call/agent-step correspondence motivating the method is described as a structural analogy rather than a formal claim (Section 2.1).

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

#### 4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 2 fully specifies the selection pipeline, scoring functions, and FIM format, with single- and multi-function selection algorithms, hyperparameters, and CoT prompt templates given in Appendix B and corpus statistics in Appendix A; Section 3.1 specifies models, post-training pipelines (R2E-Gym, SWE-Smith, SWE-Lego), evaluation protocol, and seeds.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

## 5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: We will release the 968-repository corpus and training code at camera-ready time; we do not include them with this submission to preserve double-blind anonymity. Algorithm pseudocode for both single- and multi-function selection is in Appendix B.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

## 6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 3.1 describes the mid-training and post-training setup, benchmarks, evaluation protocol (means over three seeds), and agent harness; the full hyperparameter list and token budgets are in Appendix C, and selection-pipeline thresholds, weights, and filters are in Appendix B.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

## 7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Table 1 reports means over three independent evaluation seeds with  $\pm$  standard-deviation bands on the main coding-agent benchmarks (SWE-Bench-Verified and SWE-Bench-Lite), as described in the evaluation protocol in Section 3.1.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

## 8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix C.1 reports the compute budget: all runs use a single node of 8 NVIDIA H100 80 GB GPUs, and reproducing the full set of experiments (mid-training of three base models, the three post-training pipelines and their mid-trained variants, the ablations, and the multi-seed evaluation sweeps) takes  $\approx 5,760$  GPU-hours ( $\approx 30$  days on the 8-GPU node).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

## 9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work involves no human subjects, no PII, and uses only permissively-licensed open-source code (license inventory in Appendix A.1); benchmarks used (SWE-Bench, LiveCodeBench, OJBench, FullStackBench-EN, Terminal-Bench 2.0,  $\tau$ -bench, BFCL) are publicly available academic resources.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

## 10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No]

Justification: The paper focuses on technical contributions and does not include a dedicated broader-impacts section; the work is incremental over existing code LLMs and agentic post-training pipelines, with positive impact from improved coding-agent reliability and dual-use risks comparable to those of the underlying base models, briefly touched on in Section 6.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

## 11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released artifacts are a fine-tuning corpus of permissively-licensed open-source Python code and code-completion/agent fine-tuned models that do not introduce generative capabilities for harmful content beyond what the underlying Qwen2.5-Coder/Qwen3 base models already provide; no scraped private data or high-misuse-risk content is released.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.



- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

## 12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: All base models (Qwen2.5-Coder, Qwen3), post-training pipelines (R2E-Gym, SWE-Smith, SWE-Lego, OpenHands, SWE-agent), and benchmarks (SWE-Bench, LiveCodeBench, OJBench, FullStackBench-EN, Terminal-Bench 2.0,  $\tau$ -bench, BFCL) are cited with their original publications, and the per-repository license inventory of the 968-repo corpus is in Appendix A.1.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, [paperswithcode.com/datasets](https://paperswithcode.com/datasets) has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

## 13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[No\]](#)

Justification: The 968-repository decontaminated corpus, selection pipeline, and mid-trained checkpoints are documented in Section 2.2 and Appendix A (corpus statistics, topic categories, licenses) but are not released with this submission to preserve double-blind anonymity; they will be released with full documentation at camera-ready time.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

## 14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper involves no crowdsourcing or research with human subjects; the corpus is constructed from publicly available GitHub repositories and chain-of-thought rationales are generated by an LLM (Gemini-3-Flash), not by human annotators.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

#### 15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve research with human subjects, so IRB approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

#### 16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: Section 2.4 describes the use of Gemini-3-Flash to generate chain-of-thought rationales and Gemini-3-Flash to score (CoT, completion) pairs for filtering, with the full prompt templates provided in Appendix B.9; LLMs are also used as the trained models themselves (Qwen2.5-Coder, Qwen3) per Section 3.1.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.